

Name : Sanika Jumde

Sem : V

PRN : 23070521225

Practical 11

Code:

```
practical 11 > <> index.html > ...
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4    <meta charset="UTF-8">
5    <title>Class Scheduler - Practical 11</title>
6    <style>
7      body {
8        font-family: Arial, sans-serif;
9        padding: 20px;
10       transition: background 0.3s, font-size 0.3s;
11     }
12     h2 {
13       color: #333;
14     }
15     .session {
16       padding: 10px;
17       margin: 5px 0;
18       border: 1px solid #ccc;
19       border-radius: 5px;
20       cursor: pointer;
21       transition: background 0.3s;
22     }
23     .active {
24       background: #32cd32; /* bright green for active session */
25       color: white;
26     }
27     #countdown {
28       font-weight: bold;
29       margin-top: 20px;
30     }
31     #addBtn {
32       margin-top: 15px;
33       padding: 8px 12px;
34       border: none;
35       background: #007bff;
36       color: white;
37       border-radius: 5px;
38       cursor: pointer;
39     }
40     #addBtn:hover {
41       background: #0056b3;
42     }
43   </style>
44 </head>
45 <body>
46   <h2>Class Scheduler (with Timers & Modules)</h2>
47
48   <div id="sessions">
49     <div class="session" data-time="09:00">HTML Basics - 9:00 AM</div>
50     <div class="session" data-time="10:00">CSS Design - 10:00 AM</div>
51     <div class="session" data-time="11:00">JavaScript Logic - 11:00 AM</div>
52   </div>
53
54   <button id="addBtn">Add Session</button>
55   <p id="countdown"></p>
56
57   <script type="module" src="main.js"></script>
58 </body>
59 </html>
60
```

```
practical 11 > JS main.js > ...
1  import { startCountdown, highlightActiveSession } from './timer.js';
2  import { handleViewportChange, addDynamicSessions, setupAddSessionButton } from './ui.js';
3
4  // Countdown display element
5  const countdownDisplay = document.getElementById('countdown');
6
7  // Start countdown (5 minutes)
8  startCountdown(300, countdownDisplay);
9
10 // Highlight active session every 60s
11 highlightActiveSession();
12 setInterval(highlightActiveSession, 60000);
13
14 // Handle viewport responsiveness
15 handleViewportChange();
16 window.addEventListener('resize', handleViewportChange);
17
18 // Enable event delegation and "Add Session" button
19 addDynamicSessions();
20 setupAddSessionButton();
21
```

practical 11 > JS timer.js > ...

```
1  import { formatTime } from './helper.js';
2
3  // Countdown timer function
4  export function startCountdown(duration, display) {
5      let time = duration;
6
7      const timer = setInterval(() => {
8          const minutes = Math.floor(time / 60);
9          const seconds = time % 60;
10
11          // Change color when time < 1 minute
12          display.style.color = time < 60 ? 'red' : 'black';
13          display.textContent = `Next session starts in ${formatTime(minutes, seconds)}`;
14          time--;
15
16          if (time < 0) {
17              clearInterval(timer);
18              display.textContent = 'Session Started!';
19          }
20      }, 1000);
21  }
22
23  // Highlight active session based on system time
24  export function highlightActiveSession() {
25      const sessions = document.querySelectorAll('.session');
26      const now = new Date();
27      const currentHour = now.getHours();
28      const currentMinute = now.getMinutes();
29
30      sessions.forEach(session => {
31          const [hourStr, minuteStr] = session.dataset.time.split(':');
32          const hour = parseInt(hourStr);
33          const minute = parseInt(minuteStr);
34
35          // Highlight if current time is within this session's time ± 30 minutes (for demo)
36          if (hour === currentHour && Math.abs(minute - currentMinute) < 30) {
37              session.classList.add('active');
38          } else {
39              session.classList.remove('active');
40          }
41      });
42  }
```

practical 11 > JS helper.js > ...

```
1  // Format time into mm:ss
2  export function formatTime(minutes, seconds) {
3      return `${minutes}:${seconds < 10 ? '0' + seconds : seconds}`;
4  }
5
6  // Log actions or events
7  export function logEvent(message) {
8      console.log(`[LOG] ${new Date().toLocaleTimeString()}: ${message}`);
9  }
10
```

```

practical 11 > JS ui.js > ...
1   import { logEvent } from './helper.js';
2
3   // Change background and font size based on viewport width
4   export function handleViewportChange() {
5       if (window.innerWidth < 600) {
6           document.body.style.background = '#f0f8ff';
7       } else {
8           document.body.style.background = 'white';
9       }
10
11       document.body.style.fontSize = window.innerWidth < 500 ? '14px' : '16px';
12   }
13
14   // Add dynamic behavior (event delegation)
15   export function addDynamicSessions() {
16       const sessionsDiv = document.getElementById('sessions');
17       sessionsDiv.addEventListener('click', (e) => {
18           if (e.target.classList.contains('session')) {
19               alert("You selected: " + e.target.textContent);
20               logEvent(`Clicked on: ${e.target.textContent}`);
21           }
22       });
23   }
24
25   // Add a new session dynamically (user decides name & time)
26   export function setupAddSessionButton() {
27       const button = document.getElementById('addBtn');
28       const sessionsDiv = document.getElementById('sessions');
29
30       button.addEventListener('click', () => {
31           const name = prompt("Enter session name:");
32           const time = prompt("Enter session time in HH:MM (24-hour format:");
33
34           // Validate inputs
35           if (!name || !time || !/^\d{2}:\d{2}$/.test(time)) {
36               alert("Invalid input! Please use correct format (e.g., 13:30).");
37               return;
38           }
39
40           const newSession = document.createElement('div');
41           newSession.classList.add('session');
42           newSession.dataset.time = time;
43           newSession.textContent = `${name} - ${convertTo12Hour(time)}`;
44           sessionsDiv.appendChild(newSession);
45
46           logEvent(`Added new session: ${name} at ${time}`);
47           alert(`Session "${name}" added at ${time}!`);
48       });
49   }

```

Output :

Class Scheduler (with Timers & Modules)

HTML Basics - 9:00 AM

CSS Design - 10:00 AM

JavaScript Logic - 11:00 AM

Maths - 2:30 PM

Add Session

Next session starts in 3:02

Science - 2:40 PM

History - 3:00 PM

Add Session

Next session starts in 0:49